Национальный исследовательский ядерный университет «МИФИ»

Курс «Технология программирования кибернетических систем: управление проектами» (ТПКС)

Стандарт на разработку требований к ПО

Система автоматизации управления жизненным циклом релиза ПО

Версия 2 от 13.11.2026

Состав команды

ФИО	**Группа**	**Роль**
Заранкович Дмитрий Николаевич	Б23-514	капитан
Брылкина Ксения Константиновна	Б23-544	
Поминова Александра Андреевна	Б23-514	
Пономаренко Александр Александрович	Б23-514	
Карцева Полина Алексеевна	Б23-514	
Лазарев Ярослав Олегович	Б23-514	
Деньков Даниил Станиславович	Б23-524	
Чернов Артём Вячеславович	Б23-524	

Преподаватель: Козырев В. П.

НИЯУ МИФИ, 2026

Содержание

1 Введение **3**
1.1 Цель . . . 3
1.2 Область применения . . . 3
1.3 Термины, определения и сокращения . . . 3

2 Способ описания конкретного требования **4**
2.1 Структура записи . . . 4
2.2 Правила формулировки . . . 4
2.3 Образец записи . . . 5
2.4 Запись ссылок . . . 5

3 Описание процесса **7**
3.1 Окружение процесса . . . 7
3.2 Критерий начала . . . 7
3.3 Входные данные . . . 7
3.4 Выходные данные . . . 7
3.5 Структура процесса . . . 7
3.6 Критерий окончания . . . 8
3.7 Роли и квалификация участников . . . 8

4 Правила составления спецификации **8**
4.1 Содержание спецификации . . . 8
4.2 Свойства отдельного требования . . . 9
4.2.1 Корректность . . . 9
4.2.2 Однозначность . . . 9
4.2.3 Неделимость (атомарность) . . . 9
4.2.4 Верифицируемость (тестируемость) . . . 9
4.2.5 Трассируемость . . . 9
4.3 Свойства спецификации в целом . . . 9
4.3.1 Полнота . . . 9
4.3.2 Непротиворечивость (согласованность) . . . 9
4.3.3 Неизбыточность . . . 10
4.3.4 Модифицируемость . . . 10

Лист регистрации изменений **10**

1 Введение

1.1 Цель

Стандарт устанавливает правила подготовки и проверки спецификации требований к ПО системы автоматизации управления жизненным циклом релиза. Проверяющий использует эти правила при приёмке новой редакции СПТ.

1.2 Область применения

Правила распространяются на требования высокого уровня и их изменения. Системные требования задают основание для этой работы. Архитектуру ПО и требования низкого уровня разрабатывают на следующем этапе. Процедуру разработки задаёт План разработки ПО, идентификацию версий и изменение базовых данных — План УК ПО. Основание правил — слайды В. П. Козырева «Требования», страницы 18, 20–27, 31–33 и 78.

1.3 Термины, определения и сокращения

ПО — программное обеспечение: программы и связанные с ними данные, предназначенные для выполнения задач вычислительной системой (источник: наше определение; сокращение из гайда старших курсов).

СПТ — спецификация программных требований: документ с требованиями к ПО, его входам, выходам, режимам и внешнему взаимодействию (источник: слайды 18–21; сокращение из гайда старших курсов).

ВУ — высокий уровень требований: требования к поведению и свойствам ПО, полученные из системных требований и интерфейсов (источник: слайды 17–21, 38).

НУ — низкий уровень требований: требования к компонентам ПО, получаемые при проектировании и используемые при кодировании (источник: слайды 28, 38; конспект, К:71).

Системное требование — проверяемое утверждение о поведении или свойствах системы в целом (источник: слайды 4, 17; наша формулировка).

Производное требование — требование, полученное при разработке требований или проектировании и не имеющее прямого родительского требования на уровне выше; его происхождение требует обоснования (источник: слайд 21; конспект, К:66).

Трассируемость — возможность проследить связи требования с его источниками, последующими требованиями и проверками (источник: слайды 7, 21, 36, 78).

Верификация — проверка соответствия результатов работы её исходным данным и установленным требованиям (источник: конспект, К:97, 161; наша формулировка).

Базовая версия — утверждённая версия единицы конфигурации, служащая основой последующей работы. Промежуточные версии базовыми не считаются (источник: слайды 32, 75–76).

ЗИ — запрос на изменение: описание планируемого изменения одной ЕК, порождённое утверждённым сообщением о проблеме (источник: слайды 73–74).

СП — сообщение о проблеме: запись об одном несоответствии результатов исходным данным, данных стандартам или процессов планам (источник: слайды 64–66).

УК ПО — управление конфигурацией ПО: учёт состава, версий и изменений данных жизненного цикла для сохранения целостности ПО (источник: слайды 49, 89).

ID — постоянный идентификатор требования.

2 Способ описания конкретного требования

2.1 Структура записи

Таблица 2 – Карточка требования

Поле	**Правило записи**
Идентификатор и версия	Использовать два аргумента окружения req. Вид ID и изменение версии — по Плану УК ПО. Версия документа в шапке относится ко всей СПТ.
Текст	Одно наблюдаемое обязательство ПО. При наличии условия сначала назвать событие, затем ответ системы. Ветвь «иначе» добавлять, только если она определяет исход того же решения.
Вход и выход	Назвать доступные ПО данные, их единицы, допустимые значения и наблюдаемый результат. Для события без числовых данных указать состав и состояние до и после.
Тип и статус	Указать функциональное требование, характеристику качества или проектное ограничение. Указать фактический статус: проект, на проверке либо утверждено.
Источник и связь	Поле источника содержит родительские ID либо признак производного требования с пояснением. Формат приведён ниже.
Способ проверки	Указать испытание, анализ, осмотр или демонстрацию, входные условия и критерий успеха.
Комментарий и история	Пояснения отделить от обязательного текста. В лист изменений внести дату, основание, автора, ID и новую версию изменённого требования.

2.2 Правила формулировки

Используют «ПО должно» или «система должна». Формулировка «Если …, то ПО должно …» связывает причину с наблюдаемым выходом. Для запрета указывают результат: «ПО должно отклонить запрос». Если нужно описать отсутствие воздействия, используют форму «должно не …». Слова «удобно», «быстро» и «при необходимости» заменяют проверяемым условием. Технологию указывают только как обоснованное проектное ограничение. Если значение неизвестно, записывают причину, владельца вопроса и условие снятия. До разрешения вопроса затронутую карточку сохраняют в проекте.

2.3 Образец записи

Условие примера взято из SYS_01_05: для регистрации версии необходимы все обязательные сведения о сборке. Автор заполняет окружение `req`; первый аргумент задаёт ID, второй — номер редакции. Пример предназначен для иллюстрации формата, а не для включения в реестр утверждённых требований.

Запись в исходном документе

```
\begin{req}{SRD_01_01}{1}
Если в сообщении о сборке зарегистрированного компонента
отсутствует номер версии, то ПО должно отклонить регистрацию
версии.
\reqsource{parent=[SYS_01_05]}
\reqverify{Испытание. Передать сообщение без номера версии.
Операция отклонена, новой записи версии нет.}
\reqnote{Вход: сообщение без номера версии. Выход: отказ.
Тип: функциональное. Статус: проект.}
\end{req}
```

Та же карточка после сборки документа

SRD_01_01; #1
Если в сообщении о сборке зарегистрированного компонента отсутствует номер версии, то ПО должно отклонить регистрацию версии.
Источник: parent=[SYS_01_05]
Способ проверки: Испытание. Передать сообщение без номера версии. Операция отклонена, новой записи версии нет.
Комментарий: Вход: сообщение без номера версии. Выход: отказ. Тип: функциональное. Статус: проект.

2.4 Запись ссылок

Источник записывают один раз командой `\reqsource`. Образец поля:

```
need=[П-1,П-9]
```

Имена пишут строчными латинскими буквами. Поля разделяют точкой с запятой, значения — запятой. Между полями допустимы пробелы и переносы строк. Значения перечисляют полностью, без диапазонов, повторов и пустых элементов. Порядок элементов списка не влияет на смысл. Каждое имя встречается в записи один раз.

Для SYS обязательны поля `need`, `use` и `draft` в этом порядке. Первые два поля указывают, какую потребность удовлетворяет требование и в каком сценарии оно действует. Значения П-N и ВИ-N берут из таблицы потребностей и названий сценариев документа 01, где N — положительное целое число. У требования SYS_01_04 источник записан так:

```
need=[П-1,П-9]; use=[ВИ-1]; draft=[SYS-004]
```

Поле draft указывает происхождение в части Б рабочих материалов команды: SYS-NNN — трёхзначный номер чернового требования, R-NN — двузначный номер резервной записи,

Б.N — номер раздела. Значение TEAM обозначает решение команды, основание которого фиксируют в истории документа. Так, SYS-004 указывает на черновик. В принятой спецификации регистрация имеет ID SYS_01_04.

Для SRD используют одно поле `parent`. Связь направлена от требования к ПО к каждому системному требованию, которое оно уточняет. Пример записи:

```
parent=[SYS_01_04,SYS_01_05]
```

Такая запись уместна, только если текст уточняет оба основания. Для производного требования единственная запись источника — `derived=[yes]`. В комментарии автор указывает исходное проектное допущение, причину появления требования и кому передано обоснование для рассмотрения на системном уровне.

Таблица 3 – Проверка трассировки

Проверка	**Действие**	**Критерий**
Формат записи	Сверить имена полей, разделители и типы значений с правилами выше.	Каждый ID указан полностью. В списках нет повторов и пустых элементов.
От требования к источнику	Для каждой цели найти её определение в зафиксированной редакции входного документа.	Каждой ссылке соответствует ровно одна цель нужного типа. Версию документа можно восстановить по записи УК.
От источника к требованиям	Выбрать все карточки, в которых указан данный источник. Для сценария сверить список с указателем документа 01, для системного требования — с объёмом текущей СПТ.	Все карточки, уточняющие источник, найдены. По каждому пробелу в покрытии принято решение.
Производное требование	Проверить поле derived и прочитать обоснование в комментарии.	Указаны причина, исходное допущение и передача на системный уровень. Поле parent отсутствует.

3 Описание процесса

3.1 Окружение процесса

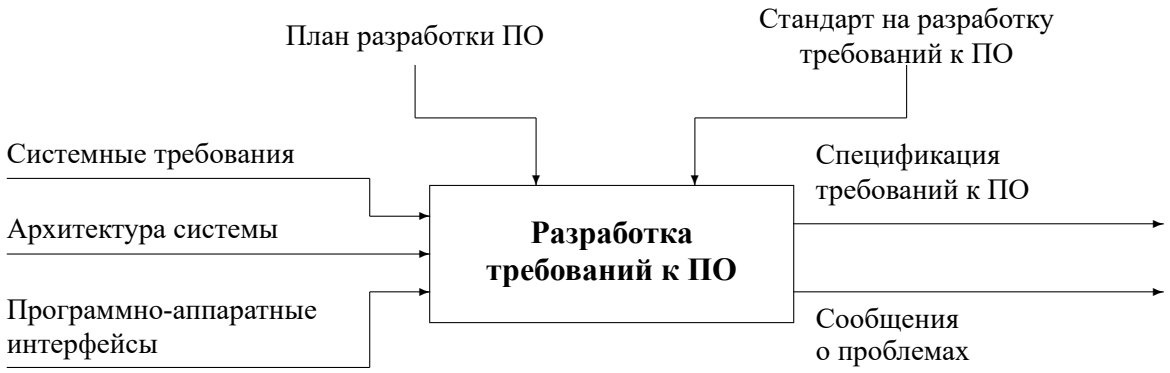


Рисунок 1 – Окружение процесса по слайду 18

При повторном входе утверждённый ЗИ определяет границы изменения базовой версии. Его рассматривают отдельно от трёх исходных данных схемы.

3.2 Критерий начала

Автор назначен, базовые версии системных требований, архитектуры и интерфейсов определены. Автору доступны применимые редакции плана и стандарта. Для изменения существующей базовой версии назначен ЗИ с указанием изменяемой ЕК.

3.3 Входные данные

Системные требования задают поведение продукта. Архитектура системы распределяет функции между системой, пользователями и внешними средствами. Программно-аппаратные интерфейсы задают границы и направление обмена. В этом проекте два последних входа представлены разделом 3 документа 01. Автор записывает редакцию используемого документа перед началом работы.

3.4 Выходные данные

Спецификация требований к ПО содержит проверяемые требования и их связи. Сообщения о проблемах описывают найденные несоответствия входов или результатов. Например, если требуемое состояние релиза невозможно определить по данным внешней системы, автор указывает в СП исходное требование и недостающее наблюдение.

3.5 Структура процесса

1. Определить входы и выходы ПО. Для каждого события записать отправителя, данные, условия допустимости и наблюдаемый ответ. Для ошибки или задержки установить, какой результат остаётся известен.
2. Сформулировать требования к ПО как к «чёрному ящику». Описать штатные и предусмотренные нештатные режимы через результаты на границе ПО. Для каждого утверждения указать способ отличить успех от ошибки.

3. Трассировать требования к породившим их системным требованиям. Для производного требования обосновать происхождение. Построить обратную выборку и разобрать каждый случай, когда системное обязательство в объёме поставки осталось без уточнения.

3.6 Критерий окончания

Независимый проверяющий сверил СПТ со входами и настоящим стандартом. Все замечания, препятствующие утверждению, закрыты. Утверждённая редакция передана в УК как базовая версия вместе с результатом проверки. Проектировщик может получить ту же редакцию по записи УК.

3.7 Роли и квалификация участников

Таблица 4 – Участники процесса разработки требований

Роль	**Что участник должен уметь**	**За что отвечает**
Аналитик	Прочитать системное требование, таблицу обмена и сценарий. Выделить событие, условия и результат.	Готовит СПТ, устанавливает связи, обосновывает производные требования, регистрирует СП.
Проверяющий	Проследить связь в обоих направлениях и применить критерий проверки к входным данным.	Независимо рассматривает редакцию и записывает результат проверки и замечания.
Ответственный за УК	Идентифицировать и восстановить редакцию документа по записи УК.	Регистрирует утверждённую версию, сохраняет результат проверки и передаёт версию проектировщику.

Проверку выполняет другой участник. Персональные назначения фиксируют по Плану разработки ПО.

4 Правила составления спецификации

4.1 Содержание спецификации

Автор описывает четыре группы сведений, указанные на слайде 20:

1. Входные и выходные данные ПО: источники, получатели, значения, единицы и условия допустимости.
2. Требования для всех режимов работы: штатное продвижение, отказ, неизвестный результат задания и восстановление после перезапуска в пределах принятых системных требований.
3. Протоколы взаимодействия ПО с аппаратурой и другими системами: последовательность сообщения и ответа, условия ошибки, повтор и ожидание. Особенности аппаратуры берут из системного окружения.
4. Ограничения на проектирование с источником и обоснованием. Например, согласованный способ аутентификации берут из архитектурного раздела системных требований.

4.2 Свойства отдельного требования

4.2.1 Корректность

Ожидаемый результат следует из исходных данных. У производного требования проверяют обоснование.

Проверка. Сравнить перечень обязательных полей с системным требованием о регистрации сборки.

4.2.2 Однозначность

Два читателя одинаково определяют событие, исполнителя и результат.

Проверка. Для фразы «после выпуска» указать точное событие: завершение установки или подтверждение выпуска.

4.2.3 Неделимость (атомарность)

Карточка задаёт одно обязательство. Если одно действие можно принять отдельно от другого, их разделяют.

Проверка. Проверить, можно ли независимо принять регистрацию версии и соблюдение предельного времени регистрации.

4.2.4 Верифицируемость (тестируемость)

Для заданных условий можно определить, выполнено ли требование. Метод проверки и критерий указаны в карточке.

Проверка. Подготовить вход без номера версии. Успех проверки означает, что операция отклонена и новая версия не появилась.

4.2.5 Трассируемость

По источнику можно найти основание, по обратной связи — уточняющие требования. Производное требование явно отмечено.

Проверка. От примера SRD_01_01 перейти к SYS_01_05 и найти ту же карточку обратной выборкой.

Величины по возможности выражают числом с единицей измерения. Для входов задают допустимые диапазоны, для вычисляемых выходов — допустимую погрешность. Проверяют совместимость с целевой аппаратурой. Детали реализации допускаются только для установленного проектного ограничения.

4.3 Свойства спецификации в целом

4.3.1 Полнота

Для каждого системного обязательства в границах СПТ определены поведение, входы, выходы и соответствующие режимы. Проверяющий разбирает пропуски обратной трассировкой.

Проверка. Для промышленного выпуска проследить оба исхода: подтверждение до окончания окна и откат при отсутствии подтверждения.

4.3.2 Непротиворечивость (согласованность)

Результаты совместимы при одинаковых исходных условиях. Например, повтор установки согласуется с правилом блокировки среды.

Проверка. Сопоставить условия повтора установки с условиями блокировки среды при неизвестном результате задания.

4.3.3 Неизбыточность

Каждое обязательство сформулировано один раз. В других разделах помещают ссылку на его ID.

Проверка. Если правило авторизации требуется в двух сценариях, проверить, что оба ссылаются на его единственную карточку.

4.3.4 Модифицируемость

ID сохраняется. По его связям можно определить затронутые карточки, изменить каждую в одном месте и воспроизвести предыдущую редакцию.

Проверка. Для изменившегося системного требования выбрать затронутые требования к ПО и восстановить их предыдущие редакции.

Лист регистрации изменений

Версия	Дата	Что изменено	Кто	Основание
1	13.11.2026	Первая версия документа	Пономаренко А. А.	Задание к паре 31.10.2026
2	13.11.2026	Введены образец записи и его печатный вид, схема окружения, таблицы участников и проверок, проверки свойств СПТ	Пономаренко А. А.	Переработка по образцам прошлых проектов